

Modernisierung Schritt 3 - Architektur, Fehlerbehandlung, Code-Qualität & Tests

Diese Dokumentation beschreibt die im Rahmen von **Schritt 3** der Modernisierungs-Roadmap umgesetzten Änderungen. Sie ergänzt die bestehende `DEPLOYMENT.md` und ändert **nichts** am Laufzeitverhalten der Anwendung – sämtliche Endpunkte, Antwortformate und Abläufe bleiben abwärtskompatibel.

Wichtig: Alle Änderungen liegen auf dem Feature-Branch `feature/step3-modernization`. Es erfolgte **kein** Merge und **kein** Deployment ohne ausdrückliche Freigabe.

1. Architektur-Modernisierung (Trennung von Daten, Logik und UI)

Der bisher monolithische `server.ts` (~1.072 Zeilen mit Bootstrap, Middleware, allen Routen und Auth-Logik in einer Datei) wurde in klar abgegrenzte Schichten aufgeteilt:

Datei	Verantwortung
<code>server.ts</code>	Nur noch Bootstrap: App erzeugen, Datenbank initialisieren, <code>listen(3000)</code> , Graceful-Shutdown (SIGTERM/SIGINT) mit <code>storage.flush()</code> .
<code>server/app.ts</code>	App-Fabrik <code>createApp({ serveStatic })</code> : Middleware-, Routen- und Fehlerbehandlungs-Verdrahtung – ohne <code>listen()</code> . Dadurch isoliert testbar (z. B. via supertest).
<code>server/http/session.ts</code>	Session-/Auth-Schicht: kapselt <code>currentUserId</code> , <code>getActorId</code> , <code>requireApiKey</code> , <code>requireAdminAuth</code> in einer <code>SessionContext</code> -Einheit.
<code>server/http/errors.ts</code>	Typisierte Fehlerklassen (siehe Abschnitt 2).
<code>server/http/asyncHandler.ts</code>	Wrapper zur zentralen Weiterleitung von Fehlern.
<code>server/http/validate.ts</code>	Request-Validierungs-Middleware auf Basis der Zod-Schemata.
<code>server/http/errorHandler.ts</code>	Zentrale Fehler-Middleware + 404-Handler für die API.

Ergebnis: Bootstrap, HTTP-/Auth-Belange und Controller-Logik sind getrennt. Die App ist ohne echten Datenbank-/Vite-Start instanziiierbar und testbar.

2. Fehlerbehandlung (robuste Validierung & aussagekräftige Meldungen)

- **Typisierte Fehlerklassen** (`server/http/errors.ts`):
`AppError`, `BadRequestError` (400), `UnauthorizedError` (401),
`ForbiddenError` (403), `NotFoundError` (404), `ConflictError` (409),
`ValidationError` (422).
- **Zentrale Fehler-Middleware** (`server/http/errorHandler.ts`): fängt alle geworfenen Fehler ab und liefert ein **einheitliches, abwärtskompatibles** Antwortformat:

json

```
{ "error": "<Meldung als Text>", "code": "<FEHLERCODE>", "details": <optional> }
```

Das Feld `error` bleibt bewusst ein **String**, weil das Frontend (`src/context/AppContext.tsx`) die Meldung direkt als Text ausliest (`data.error || 'Fehler...'`). So bleiben bestehende Clients kompatibel.

- **Validierungs-Middleware** (`server/http/validate.ts`): `validateBody(schema)` / `validateQuery(schema)` prüfen Requests gegen die vorhandenen Zod-Schemata, ersetzen den Body durch die geparsten Daten und werfen bei Fehlern eine `ValidationError` mit **feldgenauen** Details.
- **404 für unbekannte API-Routen**: `apiNotFoundHandler` liefert unter `/api/*` eine strukturierte 404-Antwort statt der SPA- `index.html` .

Angewendet u. a. auf `POST/PUT /api/time-entries` , `POST /api/working-time` , `POST /api/forecasts` .

3. Code-Qualität

- Entfernung von Duplikaten durch Auslagerung der Auth-/Session-Helfer in ein einziges Modul (zuvor als lose Closures im Monolithen).
- Konsistente Benennung und klar dokumentierte Modulgrenzen (deutschsprachige Doc-Kommentare).
- Bootstrap- und Test-Pfad nutzen dieselbe App-Fabrik – kein duplizierter Verdrahtungscode mehr.

4. Persistenz

Bereits in **Schritt 2** auf PostgreSQL umgestellt (siehe `scripts/migrate-json-to-postgres.ts` , `prisma/schema.prisma` , `server/prismaStore.ts`). In Schritt 3 unverändert.

5. Tests (Unit- & Integrationstests für die Kernlogik)

Test-Framework: **Vitest** (+ `@vitest/coverage-v8` , `supertest`).

Datei	Umfang
<code>tests/unit/holidays.test.ts</code>	20 Tests – Feiertagsberechnung, Arbeitstage, Bundesland-Auflösung
<code>tests/unit/validationSchemas.test.ts</code>	24 Tests – Zod-Validierungsschemata
<code>tests/unit/storage.test.ts</code>	8 Tests – Rate-Hierarchie-Auflösung, GoBD-Hashkette
<code>tests/integration/api.test.ts</code>	10 Tests – API via supertest (Health, strukturierte 404/422, Fehler-als-String, Clients-CRUD)

Gesamt: 62 Tests, alle grün.

Ausführen

```
npm test           # einmalig alle Tests
npm run test:watch # Watch-Modus
npm run test:coverage # mit Coverage-Report
```

Die Vitest-Konfiguration (`vitest.config.ts`) enthält ein kleines Plugin (`resolveTsFromJs`), das die im Servercode verwendeten `.js` -Importendungen auf die `.ts` -Quelldateien abbildet.

Verifikation (lokal ausgeführt)

- `npx tsc --noEmit` – fehlerfrei
- `npm test` – 62/62 Tests grün
- `npm run build` – Frontend- (Vite) **und** Server-Bundle (esbuild) erfolgreich

Nächste Schritte (nach Freigabe)

1. Review des PRs `feature/step3-modernization` .
2. Merge und Deployment gemäß `DEPLOYMENT.md` – erst nach ausdrücklicher Freigabe.